

Adaptive HACT v5: Review and Productivity-Centered Research Redesign

Review date: September 21, 2026

Research objective: reduce the cost and time required for agents to complete large software projects correctly, without increasing coordination failures or weakening acceptance criteria.

Overall assessment: major research redirection is needed. HACT v5 offers a defensible evidence-representation component, but its present experiments do not establish the productivity contribution required by this objective. Retain the verification foundation; make context lifecycle and dependency-aware execution the object of optimization.

1. Decision and scope

The most important issue is not whether the certificate tree can be made another few percent smaller. It is whether the paper optimizes a material contributor to agent cost. In v5, the check execution sequence is held fixed and the layout changes evidence serialization. The model does not change the agent's planning, repository exploration, reasoning history, number of failed patches, or allocation of work. A strong result on the paper's objective can therefore coexist with no improvement in project productivity. The manuscript acknowledges precisely this boundary [P1, Sections 3, 7, 9].

My recommendation is not to present another evidence-byte revision as an agent-productivity advance. Either publish HACT as a scoped systems component, or make it the evidence substrate of a substantially different main paper about verified progress, context retirement, and task granularity. For the user's stated objective, the second direction is the appropriate research program. A new title alone would not satisfy it.

The manuscript's scientific candor, stable-ID design, clear negative controls, and reproducible experiments should be preserved. However, correctness of an artifact and honesty about limitations do not themselves establish scientific significance or a performance advantage over strong simple baselines.

2. What the current evidence supports

Question	Evidence in v5	Review judgment
Can required hierarchical evidence be represented more compactly?	Learned balanced layouts reduce compressed bytes by 4.36-41.84% on twelve archived source candidates [P1, Table 3].	Supported for this replay cohort and representation.
Does ordinary supervisor context become smaller?	Root-status text is identical at 593-594 UTF-8 bytes [P1, Section 9.3].	No layout-induced reduction.
Does diagnostic paging become cheaper to enumerate?	The exhaustive reader needs 148 fixed-layout pages versus 95 learned-balanced pages [P1, Section 9.3].	Supported; not observed agent retrieval or reasoning.
Does the agent finish faster?	Archived harness medians are 12.98 s direct, 18.11 s strict fixed, and 18.12 s strict learned [P1, Table 9].	No demonstrated benefit; different strict-gate service has added cost.
Are LLM bills lower at matched quality?	No hosted model calls or provider usage measurements [P1, Section 9.6].	Unknown.
Does the design improve many-agent collaboration?	The HACT harness has two concurrent workers and scripted patches [P1, Appendix G.5].	Not established for autonomous large-project collaboration.

Question	Evidence in v5	Review judgment
Does it stop stale result records from authorizing publication?	Theorem 4 binds outcomes and tickets to protected identities and generations.	Conditional local evidence guarantee, not comprehensive security or race prevention.

2.1 Corrections to the accompanying assessment

Several statements in the supplied review are stronger than the paper's evidence.

First, the approximately five-second premium is measured per small three-task harness episode, not as a universal per-candidate cost. The separate five-seconds-per-candidate economic scenario is explicitly hypothetical. The 39.69/39.77-second values are summed outer-wrapper durations for the paired source checks, not isolated test-kernel timings [P1, Tables 8-9; Section 9.4].

Second, fewer diagnostic pages do not prove fewer LLM calls. No real model chose which pages to retrieve, and compact certificate packets are not a demonstrated replacement for all useful failure messages, source snippets, or stack traces.

Third, rejecting stale PASS evidence is not equivalent to preventing malicious patches, protecting a shared branch, or resolving all concurrency races. The theorem assumes an uncompromised checker, complete declared identities, protected state, and atomic local checks. The actual integration service must still compare the candidate and current integration head atomically. A file copy is not a sandbox [P1, Section 6].

Fourth, pairwise affinities are insufficient to determine certain full-wave costs. Proposition 1 explicitly does not prove that its two parity distributions have different globally optimal layouts. The stronger four-leaf marginal separation is a different result in Appendix C.

Fifth, the economic API does not call an expensive but fully measured option UNKNOWN. It returns NOT_BENEFICIAL when gain is nonpositive, UNKNOWN when required measurements are missing, and INCOMPARABLE when service identities differ [P2, `ichact/deployment.py`].

3. Fresh review audit of the archived harness records

I recomputed the nine archived per-episode records without executing a model or modifying the package. For every record, I checked that the sum and maximum of recorded HTTP message sizes equal the recorded aggregate fields. I separately checked provider-usage absence and recomputed mode-level medians. Source-file hashes are included in the audit JSON.

Mode	Archived episodes	HTTP requests	Median episode seconds	Mean prompt-content bytes	Largest prompt-content bytes
Direct pytest	3	39	12.981653	73,107.67	6,902
Strict fixed8	3	39	18.109888	71,910.67	6,541
Strict learned	3	39	18.115390	72,171.67	6,606

There are 117 requests in total, including one injected 503 per episode. Every recorded provider-usage entry is unknown. These are small scripted interactions, not a long-context stress cohort. Their input sizes cannot establish what happens when a real agent spends hundreds of turns on a coupled project.

This is a new arithmetic audit of old measurements, not a new experiment or an external replication. I did not rerun the 329-test prototype suite, the 263-test Future Code suite, or the nine harness episodes in this review. Those remain historical release claims. The included audit script reproduces the calculations directly from the supplied ZIP.

3.1 A concrete diagnostic-design gap

The inspected bridge constructs its returned card from verdict, required/executed counts, snapshot and registry/layout IDs, packet sizes, a report path, and elapsed time. It does not directly include a failing test ID, assertion location, or error signature [P2, `ichact/future_bridge.py`]. This is not necessarily wrong: a separate retrieval API could supply those details. But the scripted patch producer does not demonstrate whether a real coding agent can use that path economically. The next experiment must measure retrieval and repair, not only the size of the returned card.

3.2 Existing context controls must be attributed correctly

The vendored Future Code context builder already enforces a request-size limit, preserves mandatory task fields, limits recent observations, exposes omitted-item counts, and pages dependency/message information [P2, `vendor/future/src/future_code/context.py`]. The paper must not count these inherited controls as productivity improvements caused by learned certificate layouts. Compare against the same harness with those controls enabled.

4. The principal scientific weaknesses

4.1 The objective is misaligned with the required contribution

Certificate-byte minimization is not an estimator of accepted-project cost. It omits the actions that usually need investigation in an agent-productivity study: repeated repository discovery, duplicated context, unsuccessful patch loops, rework after interface changes, manager summarization, idle workers, and integration contention. These are candidate bottlenecks, not already established causes in this system.

A useful generic bound makes the risk clear. If the optimized subsystem accounts for fraction f of total cost and its cost is reduced by fraction r , the overall saving, with everything else held fixed, is at most $f \cdot r$. Without measuring f , a substantial local reduction has no known project-level impact. This is ordinary accounting, not an empirical estimate for HACT.

4.2 Fewer pages is a proxy, not a task outcome

A supervisor may need one precise failing assertion, not all intermediate aggregation packets. Conversely, a short status card can force repeated expensive searches if it omits the decisive evidence. Report successful localization and repair at a given total budget, not only bytes in an exhaustive hypothetical reader.

4.3 The evidence plane does not schedule the reasoning plane

The paper explicitly keeps test schedules exogenous so every layout can be scored on the same completion waves. Once an agent changes its next action in response to different context, those counterfactual trajectories are no longer observed. Replay that is valid for byte accounting cannot estimate the outcome of a different reasoning or collaboration policy. Live randomized rollouts are required.

4.4 Novelty needs comparison with the closest agent work

Clustering, balanced trees, bounded paging, durable notes, context folding, graph pruning, and decentralized task queues are not new individually. Relevant prior work now directly targets context and coordination costs. DeLM already combines decentralized task claiming with compact verified shared updates; it is particularly close to any proposed shared-capsule design [R3]. The next manuscript needs a mechanism and evidence that go beyond renaming those ingredients.

4.5 The manuscript has become a revision history

The 27-page file preserves multiple generations of algorithms and studies. This is useful for artifact provenance but distracts from the central research question. Preserve raw data and negative results in the supplement. The main paper should present one current algorithm, its nearest baselines, one primary productivity claim, and the evidence that could falsify it.

5. What the current literature implies

Primary source	Relevant lesson	Consequence for this research
The Complexity Trap, v3 [R1]	Simple observation masking is competitive with LLM summarization and substantially cheaper in its evaluated software-agent settings.	A strong cheap masking baseline is mandatory; a new controller must pay for itself.
Scaling Long-Horizon LLM Agent via Context-Folding [R2]	Branching and folding trajectories is an existing approach to context management.	Context reset or summary handoff alone is not a novel claim.
Decentralized Multi-Agent Systems with Shared Context [R3]	Parallel task claiming and compact verified shared updates already address a central-manager bottleneck.	Shared memory and a task queue alone are insufficient novelty.
Towards a Science of Scaling Agent Systems, v3 [R4]	Controlled results depend on task-architecture alignment; coordination can hurt sequential/tool-heavy work.	Vary coupling and ready-work width, not just nominal worker count.
Adaptive Graph Pruning for Multi-Agent Communication [R5]	Joint adaptation of agent count and communication topology has prior art.	A dependency-aware scheduler needs a software-specific mechanism and matched-cost evidence.
Agentless [R6]	A simple localization-repair-validation workflow is a substantive baseline.	Do not compare only with a deliberately bloated multi-agent system.
Effective harnesses for long-running agents [R7]	Persistent artifacts and incremental sessions are established engineering practice.	Persistent state is infrastructure; the research contribution must be the policy and its measured benefit.

These are contextual findings from the cited studies, not results replicated on HACT. The search is a focused related-work check, not proof of publication priority for the proposal below.

6. Proposed research center: verified-frontier context retirement

Suggested title: Verified-Frontier Agents: Context Retirement and Dependency-Aware Parallelism for Large Software Projects.

Research question: can a system retire completed reasoning from active contexts, reuse current task interfaces, and adapt task granularity so that it completes more independently accepted work per unit cost, without weakening final verification?

HACT becomes an optional evidence backend. The central mechanism is a dependency frontier that controls what working context is kept, transferred, invalidated, or reopened. The high-value hypothesis is not that shorter prompts are always better. It is that validated task boundaries can make selective forgetting and reuse safer and more economical than unconditional replay or length-only truncation.

6.1 What crosses a verified frontier

Let completed components be D and unfinished components be A. The relevant frontier contains the exported interfaces and unresolved obligations crossing from D into A, plus active global invariants. It is not the complete history of D.

A task-local capsule contains the following typed fields:

Field	Purpose
Task and requirement IDs	Preserve the externally defined objective and acceptance contract.
Candidate/head IDs and read-set bindings	Identify exactly which state the work depends on.
Write scope and interface versions	Restrict mutation and detect stale assumptions or conflicts.
Verified predicates and evidence references	State only what the trusted runner actually established.
Unresolved questions and counterexamples	Prevent inconvenient uncertainty from disappearing in summaries.
Pending messages and ownership	Preserve unfinished coordination obligations.
Artifact references, budget, and next gate	Support continuation without replaying complete histories.

Executable facts, source observations, and unverified model hypotheses must have different types. A test PASS does not certify the semantic completeness of a natural-language summary. When a capsule is too large, split the work or deliberately widen its context budget; do not truncate the acceptance contract.

6.2 Retirement, not destructive forgetting

After a meaningful local milestone, persist its artifacts and exact evidence, then evict obsolete tool output and completed reasoning from the next model request. Keep the original trace available through indexed retrieval. Other agents receive the boundary capsule only when their declared work depends on it; they do not all receive a global activity feed.

When an interface or relevant dependency changes, invalidate affected capsule facts and notify dependent tasks. Unknown dependency coverage requires conservative invalidation. Reusing an observation about unchanged source is not permission to reuse a PASS on a different integrated candidate. The existing final acceptance contract remains mandatory.

This is the connection back to HACT: stable identities can prevent stale facts from being silently recycled. But context retirement and semantic sufficiency are new policy questions and cannot be proved merely from the existing evidence-authorization theorem.

6.3 Small contexts without a single reasoning manager

Use bounded local teams and a durable dependency board. Deterministic software owns queues, leases, subscriptions, cost counters, and reporting. Local coordinators resolve interface questions within a subsystem. A root integrator sees the project contract, cross-subsystem frontier, blockers and integration evidence, not every transcript.

The amount of necessary context depends on frontier width, not simply the number of workers. A thousand independent tasks can have a small boundary; ten tightly coupled changes can have a large one. There is no unconditional constant-context guarantee for arbitrary projects. When the boundary is wide, merge coupled work, run a focused integration task, or allow a larger justified context. Eight children is a tunable engineering setting, not a universal optimum.

6.4 Adapt task granularity rather than maximize agent count

The scheduler considers continue, retrieve, retire context, split, merge, wait, cancel redundant work, and integrate. It considers a split only when write ownership is safe and predicted benefit exceeds coordination and rework costs.

For independent child tasks with enough slots, a useful diagnostic comparison is:

parallel time $\sim$ max(child times) + coordination + integration + expected rework

versus their sequential time. Shared resources, queues, model rate limits, and dependent children invalidate that simplified approximation. Fewer seconds can also cost more dollars, so the user must choose whether the objective is latency under a budget or cost at matched quality.

Initially use a cheap deterministic policy and instrument it. A learned controller is justified only if it beats that policy after including training, inference, and exploration cost. Uncertainty in a cost estimate is not resolved by naming it a confidence bound; calibration requires independent data.

7. Where a defensible new algorithmic contribution could lie

A credible contribution is the coupled decision about context ownership and parallel work: a split changes how many contexts must contain an interface, how often it is retransmitted, and how many workers need rework when it changes. A certificate-layout objective does not capture these costs.

One candidate formulation assigns each interface j a size s_j and lets $r_j(P)$ be the number of active context groups needing it under partition P . Repeated context exposure contains a term proportional to the sum of $s_j * r_j(P)$, with additional exposure after dependency invalidations. Optimization must also price local solver cost, boundary retrieval, integration, and the critical path. Independent verification may intentionally duplicate some reading and must not be classified as waste.

The research novelty would need to reside in a concrete, economical policy that uses verified frontiers to choose retirement and task boundaries under those competing costs. Stable IDs, generic graph partitioning, summarization, and simple break-even arithmetic are supporting tools, not novel contributions by themselves. Comparisons against DeLM-style verified sharing and context-folding are especially important [R2-R3].

7.1 An accounting model for context retirement

Assume N model calls, a fixed input header h , and g additional tokens retained after each call. With complete replay, submitted input is:

$$I_{full} = N * h + g * N * (N - 1) / 2.$$

Suppose N is divisible by L and context is renewed every L calls. Let s be the capsule tokens included on every call, F total extra tokens used to construct capsules, and Q all additional retrieval/rework tokens caused by renewal. Under this deliberately simplified model:

$$I_{renew} = N * (h + s) + g * N * (L - 1) / 2 + F + Q.$$

The difference is:

$$I_{full} - I_{renew} = g * N * (N - L) / 2 - N * s - F - Q.$$

This is an exact accounting identity under the stated assumptions, not a guarantee of better reasoning or lower API bills. The policy can lose when F or Q is large. Different trajectories change N ; prompt-cache prices and cache-hit behavior change monetary cost. Against a good observation-masking baseline, the quadratic history term may already be absent. The new approach then has to win through less rework, more useful retrieval, or better coordination, not this arithmetic alone.

7.2 What can and cannot be proved

Reasonable proof targets are preservation of declared obligations at handoff, rejection of stale capsule bindings, absence of lost pending messages, conditional commit soundness, bounded admission for each request, and accounting of all capsule/coordination calls. Prove optimality only for an explicitly restricted scheduling or partition class.

Do not claim that a passing local test proves a capsule contains all information needed by future agents; that shortening context always preserves success; or that a hierarchy eliminates the cost of global integration. Those are precisely where live-agent evaluation is needed.

8. Primary outcomes and causal evaluation

8.1 Optimize accepted work, not activity

The primary quality variable is whole-project acceptance under a frozen independent contract. A useful cohort metric is:

$\text{cost per accepted project} = \text{total cost of all runs} / \text{number of accepted projects}.$

The numerator includes failures, abandoned branches, retries, planning, verification, summarization, and integration. When no project passes, this quantity is undefined/infinite, not zero. Always report acceptance rate next to it; a policy must not improve its ratio by dropping hard tasks.

For latency, report deadline success and time-to-acceptance distributions with failures retained as censored or deadline-limited observations. A median over successes alone can make an unreliable method look fast. Report actual model input/output and cached-token usage, provider charges, tool compute, and wall time separately.

An operational progress signal can count fixed, independently defined milestones that remain valid at the current integration head. Count net retained progress, not repeated passes of the same check. A regressing milestone can reduce progress. This signal helps diagnose waste, but is not a substitute for final whole-project success.

8.2 Test causes rather than assume long context is the cause

Ablate context policy and coordination policy separately, then evaluate their interaction. Keep model revision, tools, acceptance service, initial snapshots, budgets and task cohort fixed. Randomize run order to reduce service-load/time confounding. Do not hold subsequent patches fixed across adaptive policies: their different actions are part of the effect to measure.

Measure repeated context exposure by artifact/version identity; long-context fraction by actual tokenizer counts; stalled iterations; interface invalidations; repair retries; integration conflicts; blocked time; peak and mean active workers; and root coordination cost. Repeated read-only review is not automatically redundant. Log its purpose.

8.3 Baselines that could falsify the proposed mechanism

Baseline	Why it is necessary
Strong single agent with observation masking and focused retrieval	Tests whether multi-agent complexity is necessary at all.
The same single agent with periodic summary/reset	Separates retirement policy from generic compaction.
Existing hierarchical Future Code with its current bounded contexts	Prevents attributing inherited harness behavior to the new method.

Baseline	Why it is necessary
Decentralized verified-board baseline	Tests novelty beyond task claiming and compact verified updates.
Proposed context retirement with a fixed scheduler	Isolates context lifecycle effects.
Proposed scheduler without frontier-aware retirement	Isolates task-allocation effects.
Full proposal with flat evidence versus HACT evidence	Determines whether certificate trees add productivity after the controller is fixed.

Use both equal total-budget comparisons and matched-quality cost comparisons. A 32-agent system must not receive 32 times the budget unnoticed. Use the same final checks, sandbox and protected state in all main arms; ordinary pytest can be a separately labeled service-cost reference.

9. Required workload and experiment design

A large repository does not automatically make one issue a large-project task. Include multi-stage changes spanning modules, shared interface updates, integration failures and dependent milestones. Independent fixes in separate repositories do not test collaborative integration.

Use a fixed public issue benchmark as an external anchor, with task-quality screening and hidden evaluation protection. SWE-bench-style patch evaluation is useful but not sufficient on its own. Long-horizon repositories and feature/refactoring tasks should be separately reported; SWE-PolyBench and SWE-Bench Pro are relevant sources, with their exact versions and acceptance-quality issues checked before use [R8-R10]. These references do not authorize access to private or proprietary task splits.

A practical staged design is a small live-model feasibility pilot followed by a frozen confirmatory cohort. Counts should follow an approved compute budget and a power analysis; a pilot is not sufficient evidence of noninferiority. For planning, a cohort on the order of 100 issue tasks plus a separate multi-stage set across several repositories is more informative than repeated runs of one three-task fixture. These are proposed sizes, not completed experiments or universal sample requirements.

On workflows with sufficient ready-work width, vary worker caps over 1, 2, 4, 8, 16 and 32; record actual concurrency. Separate protocol stress tests at larger worker counts from live-agent quality measurements. Also vary independently executable work versus strongly coupled work. The desired behavior is not maximum concurrency everywhere: it is profitable concurrency where available, with disciplined serialization where required.

Require actual model-generated edits. Scripted wrong/right patches remain useful regression fixtures but are not evidence of better reasoning. Prevent access to gold solutions and hidden evaluator state. Freeze train/validation/evaluation separation by repository or task family when learning context or scheduling policies. Cluster uncertainty estimates at the project level; repeated requests from the same run are not independent samples.

9.1 Suggested acceptance gates for the next paper

Before running the confirmatory study, select one main claim: lower total cost at a prespecified noninferiority margin, or higher accepted-project rate under a fixed cost/deadline. Also declare how latency and safety regressions will be treated.

For example, a practically meaningful cost target might be at least a 20% reduction with a clearly specified quality margin. This is a proposed go/no-go threshold, not an expected or measured result. The confidence

interval, sample-size plan, timeout handling and failed-run accounting must be fixed before seeing outcomes. Do not infer noninferiority from an insignificant difference.

No benefit over masking, verified-board and fixed-scheduler baselines means the expensive new controller should not be promoted. If HACT's tree does not add an improvement after flattening evidence while keeping the contract, keep HACT optional and say the productivity gain comes from the context/scheduling layer.

10. Concrete implementation plan

Priority	Change	Evidence required before promotion
P0	Add full inference, cache, tool-time and verification-cost telemetry; preserve unknown fields.	Reconcile provider usage and local ledgers; include every failed branch.
P0	Add task/read/write/interface bindings and indexed artifact retrieval to the board.	Reject stale facts; preserve pending messages; enforce ownership.
P1	Implement frontier capsules and evidence-triggered context retirement in the existing context builder.	Replay structural invariants, then live matched-baseline task tests.
P1	Return actionable failure capsules: failing IDs, relevant source/assertion locations, error signature and exact evidence references.	A real agent can localize and repair from them; measure retrieval cost.
P2	Add cheap split/merge/continue rules with ready-work and coupling limits.	Compare with fixed worker policies under equal budgets and quality.
P2	Keep HACT optional; avoid a new strict wrapper after every tool action.	Focused milestone checks plus unchanged independent final integration gates.
P3	Consider a learned controller only after observing an opportunity.	Outperform the deterministic policy net of learning and inference cost.

Do not delete source, acceptance inputs or irreplaceable artifacts as a cost optimization. Retain operator stop/cancel and bounded budgets. A continuously available service should wait when blocked rather than run an unproductive infinite reasoning loop.

11. Recommended manuscript structure

The new main paper should begin with a measured productivity failure, not a certificate formula. State the accepted-project objective, introduce the verified frontier and the retirement/scheduling policy, prove its narrow invariants, then present live-agent cost-quality results and their causal ablations. Explain when context can safely be retired and when coupling makes splitting counterproductive.

Keep the exact alphabetic tree, block-DP ablation, priced-window archive, serialization codecs and WAN experiments in a compact component section or supplement. Preserve all original results and scope labels. Do not retile old scripted evidence as evaluation of the new algorithm. Author/affiliation confirmation and AI-assistance disclosure remain appropriate before submission.

A defensible eventual abstract would need to contain a live-model, project-level result, such as a measured reduction in cost at matched acceptance, along with the tested scope. Until that experiment exists, label this direction a design and evaluation protocol. There is no justified replacement numerical abstract yet.

12. Final recommendation

V5 is useful as a verification-evidence subsystem. It is not yet a convincing answer to expensive, low-productivity agents on large projects. The next research effort should directly attack repeated reasoning, unnecessary context retention, inappropriate decomposition, and stale cross-agent assumptions. HACT's

identities and evidence can support that effort, but its layout optimizer cannot be credited for gains outside its intervention.

The central question becomes: **what completed work can be removed from active context, what must cross the next dependency boundary, and when is another agent actually worth its cost?** Answer that with a concrete policy and matched-quality live experiments. That is the shortest credible route from the current systems component to the user's original productivity objective.

Sources

[P1] Adaptive HACT: Service-Matched Verification Evidence and Deployment Boundaries. Version 5.0, September 21, 2026. Supplied manuscript, 27 pages. Specific sections and tables are cited in the review.

[P2] Adaptive_HACT_v5_Research_Artifact.zip. Supplied implementation and archived measurements. SHA-256: 28c3cc10ea03248371e3bfc6baebc100e0adc9fbc88c4787bea69ed6db5da3ad. Inspected paths include `ichact/future_bridge.py`, `ichact/deployment.py`, `vendor/future/src/future_code/context.py`, and `results/v3/harness/*/record.json`.

[R1] Lindenbauer et al. The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for Agent Context Management. arXiv:2508.21433v3, October 2025. <https://arxiv.org/abs/2508.21433>

[R2] Sun et al. Scaling Long-Horizon LLM Agent via Context-Folding. arXiv:2510.11967. <https://arxiv.org/abs/2510.11967>

[R3] Mao and Mirhoseini. Decentralized Multi-Agent Systems with Shared Context. arXiv:2606.10662v1, June 2026. <https://arxiv.org/abs/2606.10662>

[R4] Kim et al. Towards a Science of Scaling Agent Systems. arXiv:2512.08296v3, April 2026. <https://arxiv.org/abs/2512.08296>

[R5] Li et al. Adaptive Graph Pruning for Multi-Agent Communication. arXiv:2506.02951, June 2025. <https://arxiv.org/abs/2506.02951>

[R6] Xia et al. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489v2, October 2024. <https://arxiv.org/abs/2407.01489>

[R7] Anthropic. Effective harnesses for long-running agents. November 26, 2025. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>

[R8] Rashid et al. SWE-PolyBench: A multi-language benchmark for repository level evaluation of coding agents. arXiv:2504.08703, April 2025. <https://arxiv.org/abs/2504.08703>

[R9] Deng et al. SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks? arXiv:2509.16941, 2025. <https://arxiv.org/abs/2509.16941>

[R10] Zheng et al. SWE-Bench Pro Verified: A Reliable Benchmark for Software Engineering Agents. arXiv:2609.08149v1, September 2026. Preliminary benchmark-quality findings; not treated here as external validation of HACT. <https://arxiv.org/abs/2609.08149>

Public sources were checked September 21, 2026. Their results belong to their authors and have not been reproduced in this review.